

7/24/23, 2:12 PM

Codio - Modules and Packages

CodioProjectFileEditFindViewToolsEducationHelpRunProject Index (static)Debug

SANDIPAN-DEY

example_mo... x

```
1 def my_func():
2     print('I am a function')
3
4 class MyClass():
5     @classmethod
6     def hello(self):
7         print('I am a class')
8
9 my_string = 'I am a string'
10 my_list = ['I', 'am', 'a', 'list']
11
12 if __name__ == '__main__':
13     my_func()
```

77% (18:35)

module_revi... x Terminal

```
10 my_func()
11 MyClass.hello()
12 print(my_string)
13 print(my_list)
14
15 def my_func():
16     print('Local function definition')
17
18 my_func()
19 example_module.my_func()
```

Guide x

Collapse

Reviewing Modules

assume you already defined a function called `my_func` . Keeping the module name means you can differentiate between the two.

```
import example_module

def my_func():
    print('Local function definition')

my_func()
example_module.my_func()
```

Running Code on Import

While you do not need any special code to signify if a file is a module, you do need to be aware of some behavior specific to the importing process. Update the module (**top** panel) so that it calls `my_func` .

top panel

```
def my_func():
    print('I am a function')

class MyClass():
    @classmethod
    def hello(self):
        print('I am a class')

my_func()
```

In the **bottom** panel, simply import `example_module` ; do not do anything else. Click the button below to run your script.

bottom panel

```
import example_module
```

TRY IT

```
I am a function
I am a class
I am a string
['I', 'am', 'a', 'list']
I am a function
I am a class
I am a string
['I', 'am', 'a', 'list']
Local function definition
I am a function
```

Why did Python print that string? We often think of modules as being definitions for classes or functions. That is, modules are passive until the end user decides to implement them. However, importing a module runs **all** of the code in the file.

It is possible to keep code from being executed during the import process. Using a conditional that tests if `__name__` is equal to `__main__` will only execute associated code if Python is directly running the module file. If Python is importing the module file, then the code in the conditional is not executed.

top panel

```
def my_func():
    print('I am a function')

class MyClass():
    @classmethod
    def hello(self):
        print('I am a class')

if __name__ == '__main__':
    my_func()
```

TRY IT

```
I am a function
I am a class
I am a string
['I', 'am', 'a', 'list']
I am a function
I am a class
I am a string
['I', 'am', 'a', 'list']
Local function definition
I am a function
```

Did you notice?

Reading Question

Select the **best** definition of a module in Python.

☐ Any valid Python file that only defines functions and classes.

☐ Any valid Python file with the `@module` decorator.

☐ Any valid Python file that uses the `import` keyword.

☒ Any valid Python file that is imported by another file.

The correct answer is:

|

Any valid Python file that is imported by another file.

While modules are often classes and functions, this is not a requirement to be a module. A module is any valid Python file (no bugs, `.py` file extension, etc.). You do not need to use a decorator or any other special symbol to denote a module. Modules do not need to use the `import` key word (though they can if need be).

Check It!

Next

100% (2:1)

https://codio.com/sandipan-dey/modules-and-packages/tree/code%2Fexample_module.py

1/1